KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Loan Eligibility Verification and EMI Payment Tracking System

Submitted by

DAKSHATA S

25BIT012

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026 Semester: **II**

DECLARATION

I hereby declare that the project entitled " **Loan Eligibility Verification and EMI Payment Tracking System"** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of , Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

DAKSHATA S

CERTIFICATE

This is to certify that the project entitled Loan Eligibility Verification and EMI Payment Tracking System**”** is a **bona fide work** carried out by **[DAKSHATA S] [I-BSc IT-.’A’]**, **[]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date: 09.04.2026

Submitted for the Viva-Voce Examination held on **KG COLLAGE OF ARTS & SCIENCE**

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	

8	**References**	

ABSTRACT

The **Loan Eligibility Verification and EMI Payment Tracking System** is a Java-based application designed to automate the process of loan management. In many financial institutions and small lending environments, loan verification and EMI tracking are handled manually, which often leads to calculation errors, delays in processing, and inefficient record management. This project aims to simplify loan handling by providing an automated system that verifies eligibility and tracks EMI payments effectively.

The system allows users to enter personal and financial details, check loan eligibility based on predefined conditions, calculate EMI, and monitor payment status. It ensures that users clearly understand their repayment schedule and remaining balance, helping in better financial planning and avoiding missed payments. The application is developed using Core Java programming concepts such as classes, objects, methods, loops, conditional statements, and basic data structures.

The main objective of the system is to improve loan processing efficiency, reduce human errors, and provide accurate and timely financial information. The system displays clear loan details including loan amount, EMI, tenure, and outstanding balance, enabling better decision-making.

This project demonstrates the practical implementation of object-oriented programming concepts in Java and provides a strong foundation for developing more advanced banking and financial management systems in the future.

1. INTRODUCTION

The Loan Eligibility Verification and EMI Payment Tracking System is a Java-based console application.

It helps users determine whether they are eligible for a loan based on income.

The system also calculates the EMI using standard financial formulas.

Users can track their monthly EMI payments and remaining balance.

Manual loan calculations are often time-consuming and error-prone.

This system simplifies the entire loan management process.

It ensures accuracy, speed, and efficiency in financial calculations.

The project is useful for learning real-world Java applications.

1.1 Objective

The main objective is to automate loan eligibility checking.
It aims to calculate EMI quickly and accurately.
The system helps users track EMI payments efficiently.
It reduces manual errors in loan processing.
The project improves understanding of financial calculations.
It demonstrates the use of Java programming concepts.
It provides a simple interface for users.
Overall, it enhances decision-making in loan management.

1.2 Overview

This system is designed as a console-based application.

It collects user details such as income and loan amount.

Based on input, it verifies loan eligibility.

If eligible, EMI is calculated automatically.

Users can make EMI payments through the system.

The balance amount is updated after each payment.

The system displays loan details whenever required.

It is simple, efficient, and user-friendly.

1.3 Features

- The system verifies loan eligibility based on income.
  It calculates EMI using a predefined formula.
  It allows users to pay EMI periodically.
  It tracks remaining loan balance automatically.
  The system provides loan details on request.
  It prevents invalid loan applications.
  It is easy to use and understand.
  The system runs efficiently in a console environment.

2. PROGRAM ANALYSIS

Program analysis involves understanding the problem and defining how the system will solve it effectively.

2.1 Problem Definition

- Manual loan processing can lead to calculation errors.
  Tracking EMI payments manually is difficult.
  Users may not know their eligibility clearly.
  Banks require accurate and quick calculations.
  There is a need for an automated solution.
  The system must handle multiple operations easily.
  It should provide accurate financial results.
  Hence, a computerized system is required.

2.2 Project Overview

- The project provides a solution for loan management.
  It checks eligibility using user income details.
  It calculates EMI using mathematical formulas.
  It allows EMI payment tracking step-by-step.
  The system stores and updates loan information.
  Users can view loan status anytime.
  The project is implemented using Java.
  It is suitable for small-scale financial systems.

2.3 Scopes and Limitations

- The system is useful for basic loan calculations.
  It is suitable for educational purposes.
  It can be expanded into a web application.
  The system supports single-user operations.
  It does not include database storage.
  It lacks advanced security features.
  Internet-based features are not included.
  However, it provides a strong basic foundation.

3. SYSTEM DESIGN

System design defines how the system components are organized and interact.

3.1 Modules

The system consists of multiple modules.
The User Input Module collects user data.
The Eligibility Module verifies loan approval.
The EMI Calculation Module computes EMI.
The Payment Module handles EMI payments.
The Display Module shows loan details.
Each module works independently and efficiently.
Together, they form a complete system.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the **Loan Eligibility Verification and EMI Payment Tracking System**, a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

This section explains the steps followed to develop the system.

4.1 Problem Analysis

The first step is understanding user requirements.
The problem of manual loan tracking is analyzed.
Necessary inputs and outputs are identified.
Constraints and limitations are studied.
The system objectives are clearly defined.
Possible solutions are evaluated.
The best approach is selected.
This forms the base for development.

4.2 System Design

The system structure is planned carefully.
Modules are divided based on functionality.
Class diagrams and logic are prepared.
Input and output formats are designed.
Data flow is clearly defined.
Simple and efficient design is preferred.
User interaction is kept minimal.
This ensures smooth execution.

4.3 Program Development

The program is written using Java language.
Classes and objects are implemented.
Logic for eligibility and EMI is coded.
User input is handled using Scanner.
Conditions and loops are used effectively.
Each module is developed step-by-step.
Code is tested during development.
Errors are fixed immediately.

4.4 Testing and Validation

The system is tested with different inputs.
Valid and invalid cases are checked.
EMI calculations are verified manually.
Boundary conditions are tested.
Errors and bugs are identified.
Corrections are made to improve accuracy.
The system performance is evaluated.
Final output is validated successfully.

4.5 Result and Documentation

The system produces accurate loan results.
EMI and balance are calculated correctly.
User interaction is smooth and simple.
All modules work as expected.
Results are documented clearly.

Screenshots are included for reference.
The project report is prepared systematically.
Documentation helps in understanding the system.

4.6 Algorithm

Start the program.
Display menu options to the user.
Accept user choice.
If applying loan, collect input details.
Check eligibility conditions.
If eligible, calculate EMI.
Allow EMI payment and update balance.
Display results and repeat until exit.

4.7 Flowchart

Start → Display Menu → Get Choice
If Apply Loan → Input Details → Check Eligibility
If Eligible → Calculate EMI → Store Data
If Pay EMI → Deduct EMI from Balance
If View → Display Loan Details
Repeat process until user exits
End program after completion
This flow ensures smooth operation.

5. SOURCE CODE

There are three files in this coding there are

- Main,
- Loan, and
- LoanManager classes.

1.Main.java

```
import java.util.Scanner;
```

```
public class LoanManager {


    Loan loan;
    Scanner sc = new Scanner(System.in);


    public void applyLoan() {
        System.out.print("Enter Name: ");
        String name = sc.next();


        System.out.print("Enter Monthly Income: ");
        double income = sc.nextDouble();


        System.out.print("Enter Loan Amount: ");
        double amount = sc.nextDouble();


        System.out.print("Enter Tenure (months): ");
        int tenure = sc.nextInt();


        // Eligibility check
        if (income < 20000) {
            System.out.println("Loan Rejected: Income too low!");
            return;
        }


        if (amount > income * 20) {
            System.out.println("Loan Rejected: Amount too high!");
            return;
        }
```

```
        loan = new Loan(name, income, amount, tenure);

        System.out.println("Loan Approved!");
        System.out.println("EMI: " + loan.emi);
    }

    public void payEMI() {
        if (loan == null) {
            System.out.println("No loan found!");
            return;
        }

        if (loan.balance <= 0) {
            System.out.println("Loan already paid!");
            return;
        }

        loan.balance -= loan.emi;

        if (loan.balance < 0) loan.balance = 0;

        System.out.println("EMI Paid: " + loan.emi);
        System.out.println("Remaining Balance: " + loan.balance);
    }

    public void viewDetails() {
        if (loan == null) {
            System.out.println("No loan details available!");
            return;
```

```
        }


        System.out.println("\n--- Loan Details ---");
        System.out.println("Name: " + loan.name);
        System.out.println("Loan Amount: " + loan.loanAmount);
        System.out.println("EMI: " + loan.emi);
        System.out.println("Remaining Balance: " + loan.balance);
    }
}
```

2.Loan.java\

```
public class Loan {
    String name;
    double income;
    double loanAmount;
    double interestRate = 10; // default 10%
    int tenure; // months
    double emi;
    double balance;


    public Loan(String name, double income, double loanAmount, int
tenure) {
        this.name = name;
        this.income = income;
        this.loanAmount = loanAmount;
        this.tenure = tenure;
        this.balance = loanAmount;
        calculateEMI();
    }
```

```
    public void calculateEMI() {
        double r = interestRate / (12 * 100);
        emi = (loanAmount * r * Math.pow(1 + r, tenure)) / (Math.pow(1
+ r, tenure) - 1);
    }
}
```

3.LoanManger.java

```
import java.util.Scanner;

public class LoanManager {

    Loan loan;
    Scanner sc = new Scanner(System.in);

    public void applyLoan() {
        System.out.print("Enter Name: ");
        String name = sc.next();

        System.out.print("Enter Monthly Income: ");
        double income = sc.nextDouble();

        System.out.print("Enter Loan Amount: ");
        double amount = sc.nextDouble();

        System.out.print("Enter Tenure (months): ");
        int tenure = sc.nextInt();
```

```
        // Eligibility check
        if (income < 20000) {
            System.out.println("Loan Rejected: Income too low!");
            return;
        }

        if (amount > income * 20) {
            System.out.println("Loan Rejected: Amount too high!");
            return;
        }

        loan = new Loan(name, income, amount, tenure);

        System.out.println("Loan Approved!");
        System.out.println("EMI: " + loan.emi);
    }

    public void payEMI() {
        if (loan == null) {
            System.out.println("No loan found!");
            return;
        }

        if (loan.balance <= 0) {
            System.out.println("Loan already paid!");
            return;
        }
```

```
        loan.balance -= loan.emi;


        if (loan.balance < 0) loan.balance = 0;


        System.out.println("EMI Paid: " + loan.emi);
        System.out.println("Remaining Balance: " + loan.balance);
    }


    public void viewDetails() {
        if (loan == null) {
            System.out.println("No loan details available!");
            return;
        }


        System.out.println("\n--- Loan Details ---");
        System.out.println("Name: " + loan.name);
        System.out.println("Loan Amount: " + loan.loanAmount);
        System.out.println("EMI: " + loan.emi);
        System.out.println("Remaining Balance: " + loan.balance);
    }
}
```

6. INPUT AND OUTPUT SCREENS

```
===== LOAN SYSTEM MENU =====
1. Apply for Loan
2. Pay EMI
3. View Loan Details
4. Exit
Enter choice: 2
EMI Paid: 1758.317744600198
Remaining Balance: 18241.682255399803
```

```
===== LOAN SYSTEM MENU =====
1. Apply for Loan
2. Pay EMI
3. View Loan Details
4. Exit
Enter choice: 1
Enter Name: Arun
Enter Monthly Income: 30,000
Enter Loan Amount: 20,000
Enter Tenure (months): 12
Loan Approved!
EMI: 1758.317744600198
```

```
===== LOAN SYSTEM MENU =====
1. Apply for Loan
2. Pay EMI
3. View Loan Details
4. Exit
Enter choice: 4
Thank you!
```


7. DISCUSSION & CONCLUSION

7.1 Discussion

The system successfully automates loan processing.
It reduces manual effort and errors.
EMI calculation is accurate and fast.
Users can easily track their payments.
The system is simple yet effective.
It demonstrates practical Java usage.
Some improvements can enhance functionality.
Overall, it meets project requirements.

7.2 Conclusion

The project achieves its intended objectives.
It simplifies loan eligibility verification.
EMI tracking becomes easy and efficient.
The system is reliable and user-friendly.
It provides accurate financial calculations.
The project enhances programming knowledge.
It can be extended with advanced features.
Thus, it is a successful mini project.

8.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com